

Homework 04

*Due: August 29, 2026 at 11:59 PT**August 25, 2026*

Sources

- Book: <https://dsc40b.com/materials/default/notes/book.pdf>
- Lecture 10: https://dsc-courses.github.io/DSC40B-SU26/resources/lectures/lec10/Lecture_10_blank.pdf
- Lecture 11: https://dsc-courses.github.io/DSC40B-SU26/resources/lectures/lec11/Lecture_11_blank.pdf
- Lecture 12: https://dsc-courses.github.io/DSC40B-SU26/resources/lectures/lec12/Lecture_12_blank.pdf

Instructions

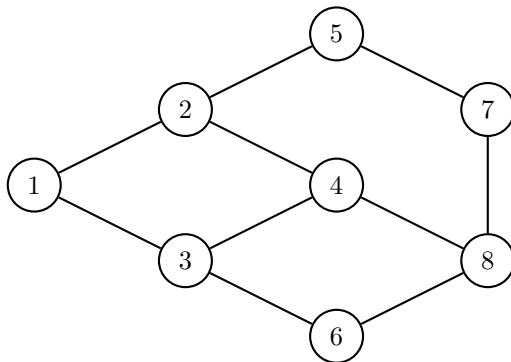
There are four problems, each worth 25 points, for a total of 100 points. For every algorithm you design, state the algorithm clearly, argue that it is correct, and give its running time in terms of $n = |V|$ and $m = |E|$ unless the problem says otherwise. Pseudocode may be written in Python. You may cite BFS, DFS, and DFS-based topological sort from lecture as black boxes with their stated running times, but you may not cite an algorithm we have not covered. Whenever a traversal has a choice of which neighbor to visit next, break the tie by taking the smallest vertex label first.

Problem 1

Walks and Paths.

[25 points]

Recall from lecture that a *walk* of length k from u to v is a sequence of vertices $w_0, w_1, \dots, w_k$ with $w_0 = u$, $w_k = v$, and $\{w_{i-1}, w_i\} \in E$ for every $i = 1, \dots, k$. A walk may repeat vertices and edges. A *path* is a walk whose vertices are all distinct. Throughout this problem, G is the undirected graph below.



The undirected graph G on $V = \{1, \dots, 8\}$.

- (a) **[3 points]** A *closed walk* is a walk whose first and last vertices are equal. Give a closed walk of length 4 in G that visits exactly two distinct vertices.

(b) Prove each of the following.

- (i) **[6 points]** If there is a walk from u to v in a graph, then there is a path from u to v , and that path has length at most the length of the walk.
- (ii) **[6 points]** Let a be the minimum length of a walk from u to v , and let b be the minimum length of a path from u to v . Use part (i) to prove that $a = b$. This is why $\text{dist}(u, v)$ may be defined using either walks or paths without ambiguity.

Hint for (i): Take a walk from u to v of minimum length and suppose it repeats a vertex. What can you delete?

(c) **[10 points]** Prove all three of the following statements about walks in an undirected graph. Each is a short argument straight from the definition of a walk.

1. For every vertex u there is a walk from u to u .
2. If there is a walk from u to v , then there is a walk from v to u .
3. If there is a walk from u to v and a walk from v to w , then there is a walk from u to w .

These three facts are what make lecture's definition of a connected component, a maximal set of vertices all reachable from one another, a sensible one: without them a vertex might fail to be reachable from itself, or reachability might run in only one direction.

Then answer this. Exactly one of your three arguments breaks if you replace every occurrence of the word "walk" with the word "path." Say which one, give a concrete example in G showing it break, and explain how part (b) repairs the statement.

Note: a relation satisfying the three statements above is called an equivalence relation. You do not need that vocabulary here, and no part of this problem requires it.

Problem 2

Representations and Traversal Orders.

[25 points]

Use the same graph G from Problem 1.

- (a) [5 points] Write down the adjacency list of G with each list sorted in increasing order, and write down the adjacency matrix of G . Then, for a general undirected graph with n vertices and m edges, state the space used by each representation in Θ notation, and give one operation that the adjacency matrix supports faster and one operation that the adjacency list supports faster. State the running time of each of those two operations under each representation.
- (b) Run BFS from vertex 1.
- (i) [2 points] Report the order in which vertices are discovered; the array of distances from vertex 1; and the set of BFS tree edges.
 - (ii) [1 points] List every edge of G that is not a BFS tree edge, together with the levels of its two endpoints.
 - (iii) [4 points] Using your answer as a guide, prove the following statement in general: if BFS is run from s on an undirected graph and $\{u, v\} \in E$, then $|\text{dist}(s, u) - \text{dist}(s, v)| \leq 1$.
- (c) [8 points] Run DFS from vertex 1, using a single counter that increments on every discovery and on every finish. Report the discovery time and the finish time of each of the eight vertices, the set of tree edges, and the set of back edges.
- Then prove the following statement in general: in a DFS of an undirected graph, every edge of G is either a tree edge or a back edge. In particular there is no edge $\{u, v\}$ such that neither of u, v is an ancestor of the other in the DFS forest.
- (d) [5 points] Prove the handshake lemma: for every undirected graph, $\sum_{v \in V} \deg(v) = 2m$. Conclude that the number of vertices of odd degree is even. Verify both statements on G .

Problem 3

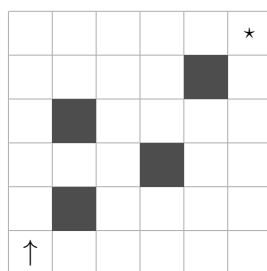
Warehouse Robot Routing.

[25 points]

A warehouse floor is an $n \times n$ grid. Each cell (i, j) with $i, j \in \{1, \dots, n\}$ is either empty or blocked by a shelf. A robot occupies one empty cell and faces one of four orientations: North, South, East, or West. At each step the robot performs exactly one of four actions:

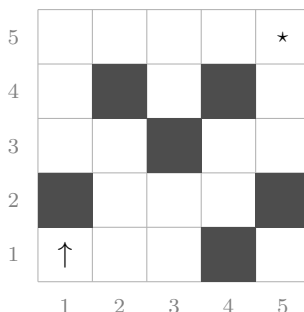
- **forward**: move one cell in the direction it faces, which is legal only if that cell is inside the grid and empty;
- **left**: stay in place and rotate the orientation 90° counterclockwise;
- **right**: stay in place and rotate the orientation 90° clockwise;
- **wait**: stay in place and keep the orientation.

A robot at (i, j) facing North moves to $(i, j + 1)$ on a forward action, and turns to face West or East on a left or right action. The robot starts at $(1, 1)$ facing North. The grid is given to you as an $n \times n$ array of booleans.



An example floor with $n = 6$. Shaded cells hold shelves. The arrow marks the start state $(1, 1)$ facing North, and the star marks the target cell (n, n) .

- [9 points] Design an algorithm that computes the minimum number of steps the robot needs to reach cell (n, n) in any orientation, or reports that (n, n) is unreachable. Describe the graph you search explicitly: say what its vertices are, what its edges are, how many of each there are, and why you do not need to build it in memory ahead of time. Argue that your algorithm is correct and show that it runs in $O(n^2)$ time.
- [4 points] Extend your algorithm so that it also outputs an actual sequence of actions of minimum length. Explain what extra bookkeeping the search performs, and explain why the reconstructed sequence is legal and has the claimed length.
- The three parts below all concern the specific floor drawn here.



The floor for part (c), with $n = 5$. Column index i runs left to right and row index j runs bottom to top, so North is up. Shelves sit at $(1, 2)$, $(2, 4)$, $(3, 3)$, $(4, 1)$, $(4, 4)$, $(5, 2)$.

- (i) **[4 points]** Prove that if the robot can reach cell (i, j) in some orientation, then it can reach (i, j) in all four orientations. Then give an $O(n^2)$ algorithm that lists every empty cell the robot can never reach from its start state, argue that it is correct, and apply it to the floor above.
- (ii) **[3 points]** Ignore orientation, and call two empty cells adjacent when they share a side. Give an $O(n^2)$ algorithm that counts the connected regions of empty cells. Your algorithm starts a traversal from some of the n^2 cells, so explain why the total work is $O(n^2)$ and not $O(n^4)$. How many regions does the floor above have?
- (iii) **[5 points]** The question now changes. A forward move takes 1 second, each turn takes 3 seconds, and waiting takes 1 second. You want the route of least total time rather than the route of fewest actions. Consider these two legal routes from the start state to $(5, 5)$, written as strings of actions where F is forward, L is left, and R is right:

$$P = \text{RFFLFRFLFRFLFF}, \quad Q = \text{RFLFFLFRFFRFFFF}.$$

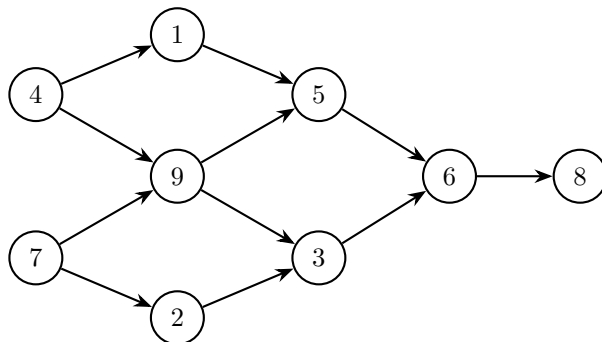
1. Report the number of actions and the total time of P and of Q . One of them is shorter in actions and the other is shorter in time. Say which is which, and state what that shows about the algorithm from part (a).
2. Name the step of the argument from lecture that layer equals distance which fails once edges carry different costs, and explain why it fails.
3. Change the graph rather than the algorithm so that plain BFS answers the least-time question. State the number of vertices of the new graph and the resulting running time.

Problem 4

Dependency Resolution with DFS.

[25 points]

A build system runs 9 tasks. Task u must finish before task v starts whenever the directed edge $u \rightarrow v$ is present. The dependency graph D is shown below.



The dependency graph D on nine tasks.

The edges of D are $4 \rightarrow 1$, $4 \rightarrow 9$, $7 \rightarrow 9$, $7 \rightarrow 2$, $1 \rightarrow 5$, $9 \rightarrow 5$, $9 \rightarrow 3$, $2 \rightarrow 3$, $5 \rightarrow 6$, $3 \rightarrow 6$, $6 \rightarrow 8$.

- (a) [5 points] Run DFS over all of D . Start at vertex 1, and restart at the smallest undiscovered vertex whenever the current search finishes. Report the discovery time and the finish time of every task. Then report the order obtained by listing the tasks in decreasing order of finish time, and verify that this order is a valid topological order by checking every one of the eleven edges of D .
- (b) Prove that the procedure in part (a) is correct in general. That is, prove both of the following.
 - (i) [4 points] If D is a directed acyclic graph, then for every edge $u \rightarrow v$ we have $f(v) < f(u)$, where f denotes the DFS finish time. Conclude that listing vertices in decreasing order of finish time gives a topological order.
 - (ii) [2 points] If a directed graph has a topological order, then it is acyclic.
- (c) [7 points] Part (b) needs a DAG, so the build system must reject an ill-formed dependency graph. Design an $O(n + m)$ algorithm that either outputs a directed cycle of D as an explicit sequence of vertices, or reports that D is acyclic. State precisely which edges your DFS treats as evidence of a cycle, prove that your test neither misses a cycle nor reports one that does not exist, and explain how the vertex sequence is recovered.

Hint: Give every vertex one of three states during the search: undiscovered, on the current recursion stack, and finished. An edge into a finished vertex is not evidence of a cycle.

- (d) [7 points] The build system can run tasks in parallel. In each round it starts every task whose dependencies have all finished in earlier rounds, and every task takes exactly one round. Define $R(v)$ to be the round in which task v runs.

Give an $O(n + m)$ algorithm that computes $R(v)$ for every v and the total number of rounds. Prove that your algorithm produces the schedule with the fewest rounds. Then apply it to D : report $R(v)$ for all nine tasks, the tasks that run in each round, and the total number of rounds.

Hint: Process the vertices in the topological order from part (a).